

Chapter 28

A Gallery of Test Matrices

*Many tricks or treats associated with the Hilbert matrix
may seem rather frightening or fascinating.*

— MAN-DUEN CHOI, *Tricks or Treats with the Hilbert Matrix* (1983)

I start by looking at a 2 by 2 matrix.

Sometimes I look at a 4 by 4 matrix.

That's when things get out of control and too hard.

*Usually 2 by 2 or 3 by 3 is enough, and I look at them,
and I compute with them, and I try to guess the facts.*

First, think of a question.

*Second, I look at examples, and then third,
guess the facts.*

— PAUL R. HALMOS²³ (1991)

*When people look down on matrices,
remind them of great mathematicians such as
Frobenius, Schur, C. L. Siegel, Ostrowski, Motzkin, Kac, etc.,
who made important contributions to the subject.*

— OLGA TAUSSKY, *How I Became a Torchbearer for Matrix Theory* (1988)

²³From interviews by Albers in [10, 1991].

Ever since the first computer programs for matrix computations were written in the 1940s, researchers have been devising matrices suitable for test purposes and investigating the properties of these matrices. In the 1950s and 1960s it was common for a whole paper to be devoted to a particular test matrix: typically its inverse or eigenvalues would be obtained in closed form.

Early collections of test matrices include those of Newman and Todd [891, 1958] and Rutishauser [1006, 1968]; most of Rutishauser's matrices come from continued fractions or moment problems. Two well-known books present collections of test matrices. Gregory and Karney [524, 1969] deal exclusively with the topic, while Westlake [1217, 1968] gives an appendix of test matrices.

In this chapter we present a gallery of matrices. We describe their properties and explain why they are useful (or not so useful, as the case may be) for test purposes. The coverage is limited, and all the matrices we consider are available in MATLAB. A comprehensive, well-documented collection of parametrized test matrices is contained in MATLAB's `gallery` function (see `help gallery`), and MATLAB has several other special matrices that can be used for test purposes (see `help elmat`).

In addition to the matrices discussed in this chapter, interesting matrices discussed elsewhere in this book include magic squares (Problem 6.4), the Kahan matrix (equation (8.11)), Hadamard matrices (§9.4), and Vandermonde matrices (Chapter 22).

Table 28.1 lists MATLAB functions that can be used to generate many of the test matrices considered in this book.

The matrices described here can be modified in various ways while still preserving some or all of their interesting properties. Among the many ways of constructing new test matrices from old are

- similarity transformations $A \leftarrow X^{-1}AX$,
- unitary transformations $A \leftarrow UAV$, where $U^*U = V^*V = I$,
- Kronecker products $A \leftarrow A \otimes B$ or $B \otimes A$ (for which MATLAB has a function `kron`),
- powers $A \leftarrow A^k$.

28.1. The Hilbert and Cauchy Matrices

The Hilbert matrix $H_n \in \mathbb{R}^{n \times n}$, with elements $h_{ij} = 1/(i + j - 1)$, is perhaps the most famous of all test matrices. It was widely used in the 1950s and 1960s for testing inversion algorithms and linear equation solvers. Its attractions were threefold: it is very ill conditioned for even moderate values of n , formulae are known for the elements of the inverse, and the matrix arises in a practical problem: least squares fitting by a polynomial expressed in the monomial basis.

Despite its past popularity and notoriety, the Hilbert matrix is not a good test matrix. It is too special. Not only is it symmetric positive definite, but it is totally positive. This means, for example, that Gaussian elimination without pivoting is guaranteed to produce a small componentwise relative backward error (as is

Table 28.1. *MATLAB functions for generating matrices discussed in this book.*

<code>compan</code>	Companion matrix
<code>frank</code>	Frank matrix—ill-conditioned eigenvalues
<code>gallery</code>	Large collection of test matrices
<code>gallery('chebspec',...)</code>	Chebyshev spectral differentiation matrix
<code>gallery('clement',...)</code>	Clement matrix—tridiagonal with zero diagonal entries
<code>gallery('kahan',...)</code>	Kahan matrix—upper trapezoidal
<code>gallery('orthog',...)</code>	Orthogonal and nearly orthogonal matrices
<code>gallery('pei',...)</code>	Pei matrix
<code>gallery('randsvd',...)</code>	Random matrix with preassigned singular values and specified bandwidth
<code>gallery('toeppen',...)</code>	Pentadiagonal Toeplitz matrix
<code>gallery('tridiag',...)</code>	Tridiagonal matrix
<code>hadamard</code>	Hadamard matrix
<code>hilb</code>	Hilbert matrix
<code>invhilb</code>	Inverse Hilbert matrix
<code>magic</code>	Magic square
<code>pascal</code>	Pascal matrix
<code>rand</code>	Matrix of uniformly distributed random numbers
<code>randn</code>	Matrix of normally distributed random numbers
<code>toeplitz</code>	Toeplitz matrix
<code>vander</code>	Vandermonde matrix

Cholesky factorization). Thus the Hilbert matrix is not a typical ill-conditioned matrix.

The (i, j) element of the inverse of the Hilbert matrix H_n is the integer

$$(-1)^{i+j}(i+j-1)\binom{n+i-1}{n-j}\binom{n+j-1}{n-i}\binom{i+j-2}{i-1}^2, \quad (28.1)$$

and

$$\det(H_n) = \frac{(1!2! \dots (n-1)!)^4}{(1!2! \dots (2n-1)!)} \sim 2^{-2n^2}. \quad (28.2)$$

There are many ways to rewrite the formula (28.1). These formulae are best obtained as special cases of those for the Cauchy matrix below.

The Cholesky factor R_n of the inverse of the Hilbert matrix is known explicitly, as is R_n^{-1} :

$$r_{ij} = \frac{\sqrt{2i-1}[(j-1)!]^2}{(i+j-1)!(j-i)!}, \quad j \geq i, \quad (28.3)$$

$$(R_n^{-1})_{ij} = (-1)^{i+j}\sqrt{2j-1}\binom{i+j-2}{i-1}\binom{j-1}{i-1}, \quad j \geq i. \quad (28.4)$$

One interesting application of these formulae is to compute the eigenvalues of H_n as the squares of the singular values of R_n ; if R_n is evaluated from (28.3) and

Table 28.2. *Condition numbers of Hilbert and Pascal matrices.*

n	$\kappa_2(H_n)$	$\kappa_2(P_n)$
2	1.9281e1	6.8541e0
3	5.2406e2	6.1984e1
4	1.5514e4	6.9194e2
5	4.7661e5	8.5175e3
6	1.4951e7	1.1079e5
7	4.7537e8	1.4934e6
8	1.5258e10	2.0645e7
9	4.9315e11	2.9078e8
10	1.6026e13	4.1552e9
11	5.2307e14	6.0064e10
12	1.7132e16	8.7639e11
13	5.6279e17	1.2888e13
14	1.8534e19	1.9076e14
15	6.1166e20	2.8396e15
16	2.0223e22	4.2476e16

the one-sided Jacobi algorithm is used to compute the singular values then high relative accuracy is obtained for every eigenvalue, as shown by Mathias [821, 1995].

The condition number of the Hilbert matrix grows at an exponential rate: $\kappa_2(H_n) \sim \exp(3.5n)$ [1141, 1954]. See Table 28.2 for the first few condition numbers (these were obtained by computing the inverse exactly using MATLAB’s Symbolic Math Toolbox [825] and then computing the norm of the numeric representation of the inverse; the numbers given are correct to the figures shown).

It is an interesting fact that the matrix $\tilde{H}_n = (1/(i + j)) \in \mathbb{R}^{n \times n}$ (a submatrix of H_{n+1}) satisfies $\mu_n := \|\tilde{H}_n\|_2 = \pi + O(1/\log n)$ as $n \rightarrow \infty$, as proved by Taussky [1131, 1949]. The convergence to π is very slow: $\mu_{200} = 2.01$, $\mu_{300} = 2.08$, $\mu_{400} = 2.12$.

That H_n^{-1} is known explicitly is not as useful a property for testing an inversion algorithm as it might appear, because H_n cannot be stored exactly in floating point arithmetic. This means that when we attempt to invert H_n we at best invert $H_n + \Delta H$ (the matrix actually stored on the computer), where $|\Delta H| \leq uH_n$, and $(H_n + \Delta H)^{-1}$ can differ greatly from H_n^{-1} , in view of the ill conditioning. A possible way round this difficulty is to start with the integer matrix H_n^{-1} , but its entries are so large that they are exactly representable in IEEE double precision arithmetic only for n less than 13.

The Hilbert matrix is a special case of a Cauchy matrix $C_n \in \mathbb{R}^{n \times n}$, whose elements are $c_{ij} = 1/(x_i + y_j)$, where $x, y \in \mathbb{R}^n$ are given n -vectors (take $x_i = y_i = i - 0.5$ for the Hilbert matrix). The following formulae give the inverse and determinant of C_n , and therefore generalize those for the Hilbert matrix. The

(i, j) element of C_n^{-1} is

$$\frac{\prod_{1 \leq k \leq n} (x_j + y_k)(x_k + y_i)}{(x_j + y_i) \prod_{\substack{1 \leq k \leq n \\ k \neq j}} (x_j - x_k) \prod_{\substack{1 \leq k \leq n \\ k \neq i}} (y_i - y_k)}$$

and

$$\det(C_n) = \frac{\prod_{1 \leq i < j \leq n} (x_j - x_i)(y_j - y_i)}{\prod_{1 \leq i, j \leq n} (x_i + y_j)},$$

the latter formula having been published by Cauchy in 1841 [207, 1841, pp. 151–159]. The LU factors of C_n were found explicitly by Cho [232, 1968]:

$$l_{ij} = \frac{x_j + y_j}{x_i + y_j} \prod_{k=1}^{j-1} \frac{(x_j + y_k)(x_i - x_k)}{(x_i + y_k)(x_j - x_k)}, \quad 1 \leq j < i \leq n,$$

$$u_{ij} = \frac{1}{x_i + y_j} \prod_{k=1}^{i-1} \frac{(x_i - x_k)(y_j - y_k)}{(x_k + y_j)(x_i + y_k)}, \quad 1 \leq i \leq j \leq n.$$

It is known that C_n is totally positive if $0 < x_1 < \dots < x_n$ and $0 < y_1 < \dots < y_n$ (as is true for the Hilbert matrix) [1133, 1962, p. 295]. Interestingly, the sum of all the elements of C_n^{-1} is $\sum_{i=1}^n (x_i + y_i)$ [743, 1997, Ex. 44, §1.2.3]. By exploiting their structure, singular value decompositions of Cauchy matrices can be computed to high accuracy; see Demmel [318, 1999].

28.2. Random Matrices

Random matrices are widely used for test purposes. Perhaps the earliest use of random matrices in numerical analysis was by von Neumann and Goldstine [1200, 1947], [501, 1951], who estimated the size of their error bounds for matrix inversion (see §9.13) for the case of random matrices; to do this, they had to estimate the condition number of a random matrix.

Intuitively, one might expect random matrices to be good at revealing programming errors and unusual behaviour of algorithms, but this expectation is not necessarily correct. For example, Miller [853, 1984, pp. 96–97] describes a mutation experiment involving Fortran codes for Gaussian elimination without pivoting, Gaussian elimination with partial pivoting, and Gauss–Jordan elimination with partial pivoting. For each code, all possible mutants were generated, where a mutant is obtained by making a single typographical change to the source code. All the mutants were tested on a single random linear system $Ax = b$, with known solution, where a_{ij} was chosen from the uniform $[-1, 1]$ distribution. Many mutants were detected by their failure to pass the test of producing a solution with forward error less than a tolerance. However, some mutants passed this test, including all those that solve a system correctly in exact arithmetic; mutants in

the latter class include those that select an incorrect pivot row and thus implement a numerically unstable algorithm. A conclusion to be drawn from Miller's experiment is that random test data can reveal some programming errors, but will not reveal all.

A good example of a problem for which random test matrices are very poor at revealing algorithmic weaknesses is matrix condition number estimation. The popular condition estimation algorithms can yield poor estimates but, in practice, never produce them for a random matrix (see Chapter 15). The role of random matrices here is to indicate the *average* quality of the estimates.

Edelman [376, 1993] summarizes the properties of random matrices well when he says that

What is a mistake is to psychologically link a random matrix with the intuitive notion of a “typical” matrix or the vague concept of “any old matrix.” In contrast, we argue that “random matrices” are very *special* matrices. The larger the size of the matrices the more predictable they are because of the central limit theorem.

Various results are known about the behaviour of matrices with elements from the normal $N(0, 1)$ distribution. Matrices of this type are generated by MATLAB's **randn** function. Let A_n denote a full $n \times n$ matrix, T_n a triangular matrix, and $\tilde{T}_n$ a unit triangular matrix, all with elements from this distribution, and let $E(\cdot)$ be the expectation operator. Then, in the appropriate probabilistic sense, the following results hold as $n \rightarrow \infty$:

$$E(\log(\kappa_2(A_n))) \approx \log n + 1.537 \quad (\text{real data}), \quad (28.5)$$

$$E(\log(\kappa_2(A_n))) \approx \log n + 0.982 \quad (\text{complex data}), \quad (28.6)$$

$$\|A_n\|_2 \approx 2\sqrt{n} \quad (\text{real data}), \quad (28.7)$$

$$\|A_n\|_2 \approx 2\sqrt{2}\sqrt{n} \quad (\text{complex data}), \quad (28.8)$$

$$\rho(A_n) \approx \sqrt{n} \quad (\text{real or complex data}), \quad (28.9)$$

$$\kappa_2(T_n)^{1/n} \approx 2 \quad (\text{real data}), \quad (28.10)$$

$$\kappa_2(\tilde{T}_n)^{1/n} \approx 1.306 \quad (\text{real data}). \quad (28.11)$$

For details of (28.5)–(28.8) see Edelman [371, 1988], and for (28.10), (28.11) see Viswanath and Trefethen [1197, 1998]. Edelman conjectures that the condition number results are true for any distribution with mean 0—in particular, the uniform $[-1, 1]$ distribution used by MATLAB's **rand**. The results (28.5) and (28.6) show that *random matrices from the normal $N(0, 1)$ distribution tend to be very well conditioned*.

The spectral radius result (28.9) has been proved as an upper bound on $\rho(A_n)$ by Geman [471, 1986] for independent identically distributed random variables a_{ij} with zero mean and unit variance, and computer experiments suggest the approximate equality for the standard normal distribution [471, 1986].

A question of interest in eigenvalue applications is how many eigenvalues of a random real matrix are real. The answer has been given by Edelman, Kostlan, and Shub [381, 1994]: denoting by E_n the expected number of real eigenvalues of an $n \times n$ matrix from the normal $N(0, 1)$ distribution,

$$\lim_{n \rightarrow \infty} \frac{E_n}{\sqrt{n}} = \sqrt{\frac{2}{\pi}}.$$

Thus the proportion of real eigenvalues, E_n/n , tends to zero as $n \rightarrow \infty$. Exact formulae for E_n for finite n are also given in [381, 1994].

As a final, and more straightforward, example of the special nature of random matrices, note that a square matrix from the uniform $[0, 1]$ distribution has all positive entries with probability 1, and therefore has a real and positive dominant eigenvalue, by the Perron–Frobenius theory [636, 1985, Chap. 8]. The remaining eigenvalues tend to be of much smaller magnitude than the dominant one [27, 1990].

28.3. “Randsvd” Matrices

By **randsvd**²⁴ we mean a matrix $A \in \mathbb{R}^{m \times n}$ formed as $A = U\Sigma V^T$, where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are random orthogonal matrices and $\Sigma = \text{diag}(\sigma_i)$ ($\sigma_i \geq 0$) is a given matrix of singular values. This type of matrix has a predetermined singular value distribution (and 2-norm condition number), but is, nevertheless, random.

Randsvd matrices have been widely used as test matrices, for example for condition estimators [582, 1987], [1070, 1980], and in the LAPACK testing software. Singular value distributions of interest include, with $\alpha := \kappa_2(A) \geq 1$ a parameter,

1. one large singular value: $\sigma_1 = 1$, $\sigma_i = \alpha^{-1}$, $i = 2:n$;
2. one small singular value: $\sigma_i = 1$, $i = 1:n-1$, $\sigma_n = \alpha^{-1}$;
3. geometrically distributed singular values: $\sigma_i = \beta^{1-i}$, $i = 1:n$, where $\beta = \alpha^{1/(n-1)}$;
4. arithmetically distributed singular values: $\sigma_i = 1 - (1 - \alpha^{-1})(i-1)/(n-1)$, $i = 1:n$.

To be precise about what we mean by “random orthogonal matrix” we specify matrices from the Haar distribution, which is a natural distribution over the space of orthogonal matrices, defined in terms of a measure called the Haar measure [874, 1982, §2.1.4]. If $A \in \mathbb{R}^{n \times n}$ has elements from the normal $N(0, \sigma^2)$ distribution and A has the QR factorization $A = QR$, with the factorization normalized so that the diagonal elements of R are positive, then Q is from the Haar distribution, for any variance σ^2 [113, 1979], [1070, 1980]. If we compute Q from this prescription, the cost is $2n^3$ flops. For our **randsvd** application, a more efficient approach is based on the following result of Stewart [1070, 1980].

Theorem 28.1 (Stewart). *Let the independent vectors $x_i \in \mathbb{R}^{n-i+1}$ have elements from the normal $N(0, 1)$ distribution for $i = 1:n$. Let $P_i = \text{diag}(I_{i-1}, \bar{P}_i)$, where $\bar{P}_i$ is the Householder transformation that reduces x_i to $r_{ii}e_1$, for $i = 1:n-1$. Then the product $Q = DP_1P_2 \dots P_{n-1}$ is a random orthogonal matrix from the Haar distribution, where $D = \text{diag}(\text{sign}(r_{ii}))$ and $r_{nn} = \text{sign}(x_n)$. $\square$*

²⁴In MATLAB matrices of this type are generated by `gallery('randsvd', ...)`.

This result allows us to compute a product form representation of a random $n \times n$ orthogonal matrix from the Haar distribution in $O(n^2)$ flops. If we implicitly compute $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ using the construction in Theorem 28.1, and then form $A = U \Sigma V^T$, exploiting the structure, the cost is $m^3 + n^3$ flops. The alternative, which involves obtaining U and V from the QR factorization of random matrices A , is about twice as expensive.

Note that forming $U \Sigma V^T$ with U and V *single* Householder matrices, as is sometimes done in the literature, is not recommended, as it produces matrices of a very special form: diagonal plus a rank-2 correction.

The construction of **randsvd** matrices is, of course, easily adapted to produce random symmetric matrices $A = Q \Lambda Q^T$ with given eigenvalues.

28.4. The Pascal Matrix

The numbers in Pascal's triangle satisfy, practically speaking, infinitely many identities.

— RONALD L. GRAHAM, DONALD E. KNUTH, and OREN PATASHNIK,
Concrete Mathematics (1994)

A particularly entertaining test matrix is the Pascal matrix $P_n \in \mathbb{R}^{n \times n}$, defined by

$$p_{ij} = \frac{(i+j-2)!}{(i-1)!(j-1)!} = \binom{i+j-2}{j-1}.$$

The rows of Pascal's triangle appear as anti-diagonals of P_n :

```
>> P = pascal(6)

P =
    1     1     1     1     1     1
    1     2     3     4     5     6
    1     3     6    10    15    21
    1     4    10    20    35    56
    1     5    15    35    70   126
    1     6    21    56   126   252
```

The earliest references to the Pascal matrix appear to be in 1958 by Newman and Todd [891, 1958] and by Rutishauser [1007, 1958] (see also Newman [890, 1962, pp. 240–241]); Newman and Todd say that the matrix was introduced to them by Rutishauser. The matrix was independently suggested as a test matrix by Caffney [196, 1963].

Rutishauser [1006, 1968, §8] notes that P_n belongs to the class of moment matrices M whose elements are contour integrals

$$m_{ij} = \int_C z^{j-1} (\bar{z})^{i-1} w(z) dz.$$

All moment matrices corresponding to a positive weight function $w(z)$ on a contour C are Hermitian positive definite (as is easily verified by considering the quadratic form $y^* M y$). The choice $C = [0, 1]$, with weight function $w(z) = 1$, yields the

Hilbert matrix. The Pascal matrix is obtained for C the circle $\{z : |z - 1| = 1\}$ and $w(z) = (2\pi i(z - 1))^{-1}$ (not $w(z) = (2\pi)^{-1}$ as stated in [1006, 1968]); the change of variable $z = 1 + \exp(i\theta)$ yields a moment integral with a positive weight function.

Remarkably, the Cholesky factor of the Pascal matrix again contains the rows of Pascal's triangle, now arranged columnwise:

```
>> R = chol(P)
```

```
R =
```

```

1      1      1      1      1      1
0      1      2      3      4      5
0      0      1      3      6     10
0      0      0      1      4     10
0      0      0      0      1      5
0      0      0      0      0      1
```

The scaled and transposed Cholesky factor $S = R^T \text{diag}(1, -1, 1, -1, \dots, (-1)^{n+1})$ is returned by `pascal(n,1)`:

```
>> S = pascal(6, 1)
```

```
S =
```

```

1      0      0      0      0      0
1     -1      0      0      0      0
1     -2      1      0      0      0
1     -3      3     -1      0      0
1     -4      6     -4      1      0
1     -5     10    -10      5     -1
```

It is involutory: $S^2 = I$. This special property leads us to several more properties of $P = P_n$. First, since $P = SS^T$, $P^{-1} = S^{-T}S^{-1} = S^TS$, and so P^{-1} has integer entries (as is also clear from the fact that $\det(P) = \det(R)^2 = 1$). Moreover,

$$P = SS^T = S(S^TS)S^{-1} = SP^{-1}S^{-1},$$

so P and P^{-1} are similar and hence have the same eigenvalues. In other words, the eigenvalues appear in reciprocal pairs. In fact, the characteristic polynomial π_n has a palindromic coefficient vector, which implies the reciprocal eigenvalues property, since $\pi_n(\lambda) = \lambda^n \pi_n(1/\lambda)$. This is illustrated as follows (making use of MATLAB's Symbolic Math Toolbox):

```
>> poly(sym(P))
```

```
ans =
```

```
1-351*x+6084*x^2-13869*x^3+6084*x^4-351*x^5+x^6
```

```
>> eig(P)
```

```
ans =
```

0.0030
 0.0643
 0.4893
 2.0436
 15.5535
 332.8463

Since P is symmetric, its eigenvalues are its singular values and so we also have that $\|P\|_2 = \|P^{-1}\|_2$ and $\|P\|_F = \|P^{-1}\|_F$. Now

$$p_{nn} \leq \|P\|_2 \leq (\|P\|_1 \|P\|_\infty)^{1/2} = \left(\frac{2n-1}{n}\right) p_{nn},$$

where for the last equality we used a binomial coefficient summation identity from [517, 1994, §5.1]. Hence, using Stirling's approximation ($n! \sim \sqrt{2\pi n}(n/e)^n$),

$$\kappa_2(P_n) \sim \left(\frac{(2n)!}{(n!)^2}\right)^2 \sim \frac{16^n}{n\pi}.$$

Thus P_n is exponentially ill conditioned as $n \rightarrow \infty$.

It is worth pointing out that it is not hard to generate symmetric positive definite matrices with determinant 1 and the reciprocal root property. Let $X = ZDZ^{-1}$ where Z is nonsingular and $D = \text{diag}(\pm 1) \neq \pm I$. Then $X^2 = I$ and the matrix $A = X^T X$ has the desired properties. If we choose Z lower triangular then X is the Cholesky factor of A up to a column scaling by $\text{diag}(\pm 1)$.

The inverse of the Pascal matrix was found explicitly by Cohen [258, 1975]: the (i, j) element is

$$(-1)^{i-j} \sum_{k=0}^{n-i} \binom{i+k-1}{k} \binom{i+k-1}{i+k-j}, \quad i \geq j.$$

The Pascal matrix can be made singular simply by subtracting 1 from the (n, n) element. To see this, note that

$$\begin{aligned} P - e_n e_n^T &= S S^T - e_n e_n^T = S(I - (S e_n)(S e_n)^T) S^T \\ &= S \text{diag}(1, 1, \dots, 1, 0) S^T. \end{aligned}$$

This perturbation, $\Delta P = -e_n e_n^T$, is far from being the smallest one that makes P singular, which is $\Delta P_{\text{opt}} = -\lambda_n v_n v_n^T$, where λ_n is the smallest eigenvalue of P and v_n is the corresponding unit eigenvector, for $\|\Delta P_{\text{opt}}\|_2 = \lambda_n = \|P^{-1}\|_2^{-1}$ is of order $(n!)^2/(2n)! \sim 4^{-n} \sqrt{n\pi}$, as we saw above.

A more subtle property of the Pascal matrix is that it is totally positive. Karlin [712, 1968, p. 137] shows that the matrix with elements $\binom{i+j-1}{j}$ ($i, j = 0, 1, \dots$) is totally positive; the Pascal matrix is a submatrix of this one and hence is also totally positive. From the total positivity it follows that the Pascal matrix has distinct eigenvalues, which (as we already know from the positive definiteness) are real and positive, and that its i th eigenvector (corresponding to the i th largest eigenvalue) has exactly $i - 1$ sign changes [454, 1959, Thm. 13, p. 105].

$T = \text{pascal}(n, 2)$ is obtained by rotating S clockwise through 90 degrees and multiplying by -1 if n is even:

```
>> T = pascal(6, 2)
```

```
T =
```

```

-1    -1    -1    -1    -1    -1
 5     4     3     2     1     0
-10    -6    -3    -1     0     0
 10     4     1     0     0     0
 -5    -1     0     0     0     0
  1     0     0     0     0     0
```

It has the surprising property that it is a cube root of the identity, a property noted by Turnbull [1167, 1929, p. 332]:

```
>> T*T
```

```
ans =
```

```

 0     0     0     0     0     1
 0     0     0     0    -1    -5
 0     0     0     1     4    10
 0     0    -1    -3    -6   -10
 0     1     2     3     4     5
-1    -1    -1    -1    -1    -1
```

```
>> T*T*T
```

```
ans =
```

```

 1     0     0     0     0     0
 0     1     0     0     0     0
 0     0     1     0     0     0
 0     0     0     1     0     0
 0     0     0     0     1     0
 0     0     0     0     0     1
```

Finally, we note that it is trivial to plot an approximation to the Sierpinski gasket [931, 1992, §2.2], [518, 1992] in MATLAB: simply type `spy(rem(pascal(n), 2))`. See Figure 28.1. The picture produced by this command is incorrect for large n , however, because the elements of `pascal(n)` become too large to be exactly representable in floating point arithmetic.

28.5. Tridiagonal Toeplitz Matrices

A tridiagonal Toeplitz matrix has the form

$$T_n(c, d, e) = \begin{bmatrix} d & e & & & \\ c & d & \ddots & & \\ & \ddots & \ddots & e & \\ & & c & d \end{bmatrix}.$$

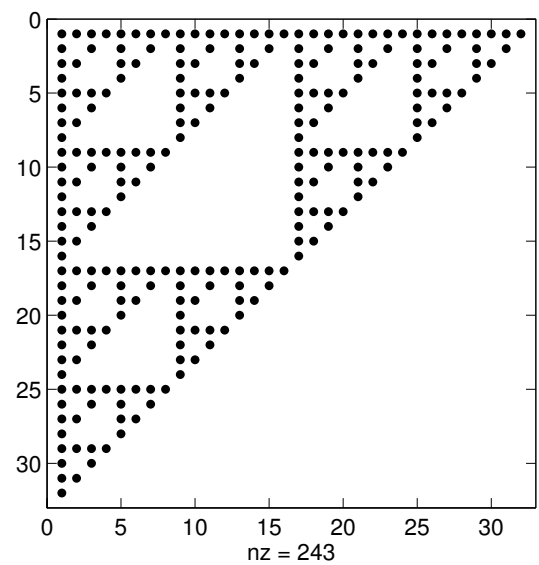


Figure 28.1. `spy(rem(pascal(32),2))`.

Such matrices arise, for example, when discretizing partial differential equations or boundary value problems for ordinary differential equations. The eigenvalues are known explicitly [1004, 1947], [1005, 1952], [1143, 1977, pp. 155–156]:

$$d + 2(ce)^{1/2} \cos(k\pi/(n + 1)), \quad k = 1:n.$$

The eigenvalues are also known for certain variations of the symmetric matrix $T_n(c, d, c)$ in which the $(1, 1)$ and (n, n) elements are modified; see Gregory and Karney [524, 1969].

The matrix $T_n(-1, 2, -1)$ is minus the well-known second difference matrix, which arises in applying central differences to a second derivative operator. Its inverse has (i, j) element $-i(n - j + 1)/(n + 1)$ (cf. Theorem 15.9). The condition number satisfies $\kappa_2(T_n) \sim (4/\pi^2)n^2$.

One interesting property of $T_n(c, d, e)$ is that the diagonals of its LU factorization converge as $n \rightarrow \infty$ when T_n is symmetric and diagonally dominant, and this allows some savings in the computation of the LU factorization, as shown by Malcolm and Palmer [810, 1974]. Similar properties hold for cyclic reduction; see Bondeli and Gander [150, 1994].

28.6. Companion Matrices

The companion matrix associated with the characteristic polynomial

$$\det(A - \lambda I) = (-1)^n(\lambda^n - a_{n-1}\lambda^{n-1} - \dots - a_0)$$

of $A \in \mathbb{C}^{n \times n}$ is the matrix

$$C = \begin{bmatrix} a_{n-1} & a_{n-2} & \cdots & \cdots & a_0 \\ 1 & 0 & \cdots & \cdots & 0 \\ 0 & 1 & \ddots & & 0 \\ \vdots & & \ddots & 0 & 0 \\ 0 & \cdots & \cdots & 1 & 0 \end{bmatrix}.$$

In MATLAB C can be computed from A via $\mathbf{C} = \text{compan}(\text{poly}(\mathbf{A}))$. It is easy to check that C has the same characteristic polynomial as A , and that if λ is an eigenvalue of C then $[\lambda^{n-1}, \lambda^{n-2}, \dots, \lambda, 1]^T$ is a corresponding eigenvector. Since $C - \lambda I$ has rank at least $n - 1$ for any λ , C is nonderogatory, that is, in the Jordan form no eigenvalue appears in more than one Jordan block. It follows that A is similar to C only if A is nonderogatory.

There are no explicit formulae for the eigenvalues of C , but, perhaps surprisingly, the singular values have simple representations, as found by Kenney and Laub [725, 1988] (see also Kittaneh [736, 1995]):

$$\begin{aligned} \sigma_1^2 &= \frac{1}{2} \left(\alpha + \sqrt{\alpha^2 - 4a_0^2} \right), \\ \sigma_i^2 &= 1, \quad i = 2:n-1, \\ \sigma_n^2 &= \frac{1}{2} \left(\alpha - \sqrt{\alpha^2 - 4a_0^2} \right), \end{aligned}$$

where $\alpha = 1 + a_0^2 + \cdots + a_{n-1}^2$. These formulae generalize to block companion matrices, as shown by Higham and Tisseur [625, 2001].

The `compan` function is a useful means for generating new test matrices from old. For any $A \in \mathbb{C}^{n \times n}$, `compan(poly(A))` is a nonnormal matrix with the same eigenvalues as A (to be precise, `compan(poly(A))` is normal if and only if $a_0 = 1$ and $a_i = 0$ for $i > 0$).

Companion matrices tend to have interesting pseudospectra, as illustrated in Figure 28.2. For more information on the pseudospectra of companion matrices see Toh and Trefethen [1144, 1994].

28.7. Notes and References

The earliest reference to the Hilbert matrix appears to be [626, 1894], wherein Hilbert obtains the formula (28.2) for $\det(H_n)$.

The formula (28.1) for H_n^{-1} is taken from Choi [233, 1983], who describes various interesting properties of the Hilbert matrix and its infinite analogue. An excellent reference for the derivation of formulae for the inverse of the Cauchy and Hilbert matrices is Knuth [743, 1997, pp. 37–38]. Another reference for the Cauchy matrix is Tyrtshnikov [1172, 1991]. The formulae (28.3) and (28.4) for the Cholesky factor and its inverse are from Choi [233, 1983] and Todd [1141, 1954].

Forsythe and Moler [431, 1967] have a chapter devoted to the Hilbert matrix, in which they describe the underlying least squares problem and discuss numerical

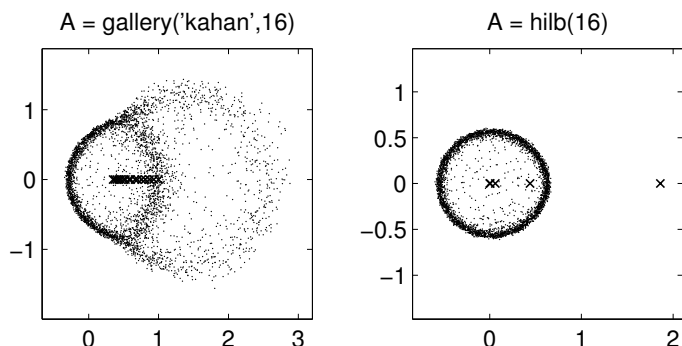


Figure 28.2. *Pseudospectra of `companion(poly(A))`.*

computation of the inverse. There have been many papers on the Hilbert matrix; two of particular interest are by Todd [1141, 1954], [1142, 1961].

Other references on the eigenvalues and condition numbers of random matrices include Edelman [372, 1991], [375, 1992] and Kostlan [748, 1992].

Anderson, Olkin, and Underhill [25, 1987] suggest another way to construct random orthogonal matrices from the Haar distribution, based on products of random Givens rotations.

The involutory triangular matrix `pascal(n,1)` arises in the stepsize changing mechanism in an ordinary differential equation code based on backward differentiation formulae; see Shampine and Reichelt [1032, 1997].

We mention some other collections of test matrices. The Harwell–Boeing collection of sparse matrices, largely drawn from practical problems, is presented by Duff, Grimes, and Lewis [362, 1989], [363, 1992]. Bai, Day, Demmel, and Dongarra [44, 1997] present a collection of test matrices for large-scale nonsymmetric eigenvalue problems. Zielke [1281, 1986] gives various parametrized rectangular matrices of fixed dimension with known generalized inverses. Demmel and McKenney [332, 1989] present a suite of Fortran 77 codes for generating random square and rectangular matrices with prescribed singular values, eigenvalues, band structure, and other properties. This suite is part of the testing code for LAPACK (see below), and MATLAB’s `gallery('randsvd',...)` is modeled on it.

The Web repository *Matrix Market* [147, 1997] at <http://math.nist.gov/MatrixMarket/> contains a large collection of test matrices, comprising both individual matrices of fixed dimension (usually large and sparse, coming from applications) and matrix generators that can produce matrices of arbitrary dimension. The matrices can be downloaded in a variety of formats. Matrix Market includes matrices from several of the collections mentioned above.

In testing matrix algorithms it can be useful to be able to construct matrices with some given combination of eigenvalues, singular values, and diagonal elements (subject to any necessary constraints on the given data). Algorithms of this type can be summarized as follows.

- *Construct symmetric A with given diagonal elements and given eigenvalues.*
See Chan and Li [209, 1983], [210, 1983] and Zha and Zhang [1280, 1995]

(see also Ikramov [659, 1998]). A special case is the generation of random correlation matrices with specified eigenvalues (a correlation matrix is a symmetric positive semidefinite matrix with ones on the diagonal); see Davies and Higham [293, 2000] and the references therein.

- *Construct A with given eigenvalues and given singular values.* See Chu [238, 2000], and see Kosowski and Smoktunowicz [747, 2000] for the case of triangular matrices with unit diagonal (the latter paper solves a research problem posed in the first edition of this book).
- *Construct A with given singular values and given diagonal elements.* See Chu [237, 1999].

Note that not all these algorithms come with a guarantee that the construction is numerically stable: caveat emptor!

28.7.1. LAPACK

The LAPACK distribution contains a suite of routines for generating test matrices, located in the directory `LAPACK/TESTING/MATGEN` (in Unix notation). These routines (whose names are of the form `xLAzzz`) are used for testing when LAPACK is installed and are not described in the *LAPACK Users' Guide* [20, 1999].

Problems

28.1. Investigate the spectral and pseudospectral properties of pentadiagonal Toeplitz matrices. See Figure 28.3. MATLAB's `gallery('toeppen',...)` generates matrices of this type. References are Beam and Warming [96, 1993] and Reichel and Trefethen [978, 1992].

28.2. (RESEARCH PROBLEM) Two methods for generating a real orthogonal matrix from the Haar distribution are Stewart's method (Theorem 28.1), based on Householder transformations, and a method of Anderson, Olkin, and Underhill [25, 1987], based on Givens rotations. Compare the efficiency of these two methods when used to generate `randsvd` matrices. Investigate the use of the Givens rotations approach to construct random banded matrices with given singular values and random symmetric banded matrices with given eigenvalues, and compare with the technique of generating a full matrix and then using band reduction (as implemented in MATLAB's `gallery('randsvd',...)`).

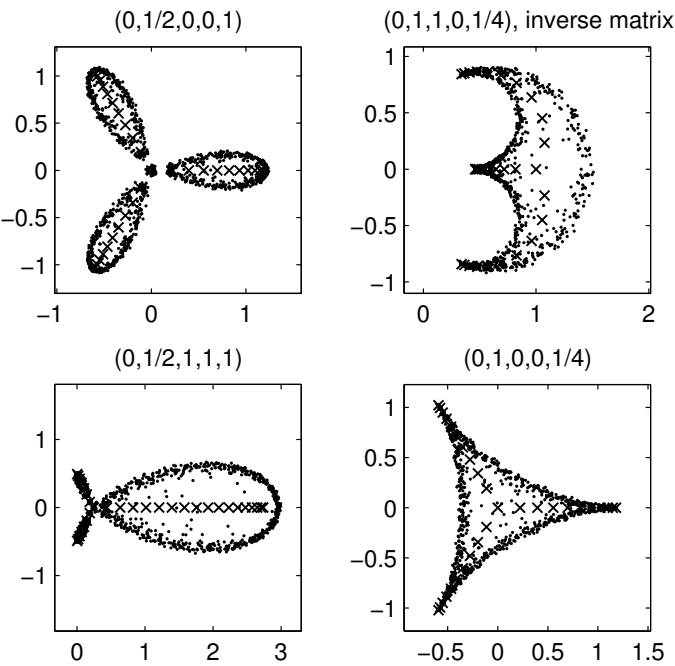


Figure 28.3. *Pseudospectra of 32×32 pentadiagonal Toeplitz matrices, `gallery('toeppen',32,a,b,c,d,e)`. Shown are the parameters (a,b,c,d,e) .*